

Java

Constructor, multithreading, file handling

```
public class Constructor {

    String name;
    int age;

    Constructor(){
        System.out.println("Constructor is called");
    }

    Constructor(String name , int age){
        this.name = name;
        this.age = age;
    }

    Constructor(Constructor obj){
        this.name = obj.name;
        this.age = obj.age;
    }

    public static void main(String[] args) {

        Constructor obj = new Constructor("Fiit", 24);

        Constructor obj1 = new Constructor(obj);

        System.out.println(obj1.age);
        System.out.println(obj1.name);

        // int data =10;

        // int data = 10;

        // data = "fiit";

        // java => static typed language

        // System.out.println(data);
```

```

        // default constructor
        // parameterized constructor
    }
}

```

```

//thread class
class myThread extends Thread{
    public void run(){ //override
        System.out.println("Thread running");
        // System.out.println(Thread.currentThread().getName());
    }
}

// 2. runnable interface

class myTask implements Runnable{
    public void run(){
        System.out.println("Runnable running");
        // System.out.println(Thread.currentThread().getName());
    }
}

public class Multithreading {
    public static void main(String[] args) throws InterruptedException {

        myThread t1 = new myThread();
        t1.setName("First thread");
        t1.start(); // thread running

        System.out.println(t1.isAlive());

        myTask interThread = new myTask();
        Thread t2 = new Thread(interThread);
        t2.setName("Second thread");
        t2.start();

        t2.setPriority(Thread.MAX_PRIORITY);

        // Thread.sleep(10000);

        t1.join();
    }
}

```

```

        t2.join();

        System.out.println("Multithreading");

        System.out.println(t2.isAlive()); //return false

        // multitasking -> chrome, whatsapp , microsoft word

        // chrome -> single process has multiple threads

        // smallest unit inside process

        // how to create threads -> two ways -> thread class, runnable
interface

    }

}

class A extends Thread{
    public void run(){
        for(int i=1; i<=5; i++){
            System.out.println("A : "+i);
        }
    }
}

class B extends Thread{

    public void display(){
        System.out.println("Hello b class");
    }
    public void run(){
        for(int i=1; i<=5; i++){
            System.out.println("B : "+i);
        }
    }
}

public class MultiPractical {

    public static void main(String[] args) {

        A t1 = new A();
        B t2 = new B();

        t1.start();
        t2.start();
    }
}

```

```
}  
  
}
```

```
class Counter{  
    int count =0;  
  
    synchronized void increment(){  
        count++;  
    }  
}  
  
class Bank{  
    int balance = 1000;  
  
    synchronized void withdraw(int amount){  
        if(balance >= amount){  
            balance -= amount;  
            System.out.println("Withdraw successfully");  
        }else{  
            System.out.println("insufficient balance");  
        }  
    }  
}  
  
public class multiSynchornized {  
  
    public static void main(String[] args) throws InterruptedException{  
  
        Counter c = new Counter();  
  
        // c.increment();  
        // c.increment();  
  
        // lamda expresssion  
        Thread t1 = new Thread( ()->{  
            for(int i=1; i<=15000; i++){  
                c.increment();  
            }  
        } );  
  
        Thread t2 = new Thread( ()->{  
            for(int i=1; i<=15000; i++){
```

```

        c.increment();
    }
} );

Thread t3 = new Thread( ()->{
    for(int i=1; i<=15000; i++){
        c.increment();
    }
} );

t1.start();
t2.start();
t3.start();

t1.join();
t2.join();
t3.join();

System.out.println(c.count);

}
}

```

```

import java.io.File;
import java.io.FileWriter;
import java.io.IOException;
import java.util.Scanner;

public class Filehandling {
    public static void main(String[] args) throws IOException {

        File folder = new File("E:\\Fiiit\\fileDemo");

        folder.mkdir();

        File file = new File("E:\\Fiiit\\fileDemo\\Demo1\\Demo2\\Demo3");

        file.mkdirs();
    }
}

```

```
        File files = new
File("E:\\Fiiit\\fileDemo\\Demo1\\Demo2\\Demo3\\demo.txt");

        files.createNewFile();

        File files1 = new
File("E:\\Fiiit\\fileDemo\\Demo1\\Demo2\\Demo3\\demo.pdf");

        files1.createNewFile();

        File files2 = new
File("E:\\Fiiit\\fileDemo\\Demo1\\Demo2\\Demo3\\demo.doc");

        files2.createNewFile();

        File files3 = new
File("E:\\Fiiit\\fileDemo\\Demo1\\Demo2\\Demo3\\demo.ppt");

        files3.createNewFile();


        System.out.println(files2.getName());

        System.out.println(files2.getAbsolutePath());

        System.out.println(files2.length());

        files2.delete();


        if(files2.exists()){
            System.out.println("demo.doc file is exist");
        }

        file.createNewFile();


        FileWriter writer = new
FileWriter("E:\\Fiiit\\fileDemo\\Demo1\\Demo2\\Demo3\\demo.txt", true);

        // writer.write("Hello, This is sample file.. \n Hello, this is sample
file ");

        writer.write("this is second line ");

        writer.close();
```

```
Scanner sc = new Scanner(files);

while(sc.hasNextLine()){
    String data = sc.nextLine();
    System.out.println(data);
}

sc.close();
}
}
```

Theory Notes

1. Constructors

- Special methods used to initialize objects.
- Types:
 - **Default constructor** → no parameters.
 - **Parameterized constructor** → accepts arguments.
 - **Copy constructor** → initializes object using another object.
- Automatically called when object is created.

2. Multithreading

- **Definition:** Running multiple threads (smallest unit of a process) concurrently.
- Ways to create threads:
 - Extend Thread class and override run().
 - Implement Runnable interface and pass to Thread.
- Methods:
 - start() → begins execution.
 - join() → waits for thread to finish.
 - setPriority() → sets thread priority.
 - isAlive() → checks if thread is running.

3. Synchronization

- Ensures **thread safety** when multiple threads access shared resources.
- Achieved using synchronized keyword.
- Prevents race conditions.

4. File Handling

- Classes: File, FileWriter, Scanner.
- Operations:
 - `createNewFile()` → creates file.
 - `mkdir()` / `makedirs()` → create directories.
 - `delete()` → removes file.
 - `write()` → writes to file.
 - Scanner → reads file content.

Coding Round Questions

1. Create a class with default, parameterized, and copy constructors. Demonstrate object creation using all three.
2. Write a program to create a thread by extending Thread class and another by implementing Runnable.
3. Demonstrate multithreading by printing numbers from two threads simultaneously.
4. Write a program with a shared counter variable incremented by multiple threads. Use synchronized to ensure correct output.
5. Create a Bank class with synchronized `withdraw()` method. Simulate multiple threads withdrawing money.
6. Write a program to create a folder and a file inside it. Print file name, path, and size.
7. Write a program to write text into a file and then read it back using Scanner.
8. Demonstrate deleting a file and checking if it exists.